Bookflow

MANUAL TÉCNICO

PROYECTO INTERMODULAR

Alba Barroso Fernández
DAW 2ºA

Índice

1. Introducción **3**
2. Arquitectura de la aplicación **4**
2.1 Frontend 5
2.1.1 Tecnologías usadas 5
2.1.2 Justificación de la elección 6
2.1.3 Entorno de desarrollo 7
2.2 Backend 7
2.2.1 Tecnologías usadas 8
2.2.2 Justificación 8
2.2.3 Entorno de desarrollo 9
3. Documentación Técnica **9**
3.1 Análisis 9
3.2 Desarrollo 12
3.3 Pruebas realizadas 15
4. Proceso de despliegue **16**
4.1 Software necesario 16
4.2 Pasos para desplegar la aplicación 17
5. Propuestas de mejoras **19**

1. Introducción

BookFlow es una aplicación web desarrollada como proyecto final para los ciclos de Técnico Superior en Desarrollo de Aplicaciones Web y Multiplataforma. Su finalidad es ofrecer a cada usuario un entorno unificado donde pueda gestionar su biblioteca personal, descubrir nuevas lecturas y compartir reseñas literarias, con una experiencia de uso sencilla, visual y accesible desde cualquier dispositivo con navegador moderno.

La aplicación sigue una arquitectura *front-only*: toda la lógica de presentación y de interacción reside en el cliente (React + Vite + TailwindCSS) y los servicios de autenticación, base de datos y almacenamiento de ficheros se delegan en Supabase, que actúa como *backend-as-a-service* (BaaS). De este modo se simplifica la infraestructura, se reduce el número de servicios a desplegar y se facilita el mantenimiento del proyecto.

A nivel funcional, **BookFlow** permite:

- Registro, inicio, cierre de sesión y recuperación de contraseñas de usuarios mediante Supabase Auth.

- Gestión de un perfil con datos básicos (nombre, correo), biografía y avatar.

- Visualización de un catálogo general de libros interactivo, optimizado para navegación táctil y de escritorio.

- Gestión de la biblioteca personal, permitiendo clasificar libros en listas ("Pendientes", "Leyendo", "Terminados").

- Sistema social para redactar, editar y eliminar reseñas y puntuaciones (1 a 5 estrellas) sobre las obras leídas.
- Un panel de administración donde el rol *admin* puede gestionar el catálogo global de libros (CRUD) y supervisar o desactivar cuentas de usuario.

La motivación principal del proyecto es cubrir la necesidad de disponer de una herramienta moderna y ligera que mejore las interfaces sobrecargadas de las plataformas actuales de lectura, ayudando a los usuarios a organizar su hábito literario de forma intuitiva.

2. Arquitectura de la aplicación

La arquitectura de **BookFlow** está basada en una Single Page Application (SPA) desarrollada con React. El proyecto se organiza en las siguientes capas:

- **Capa de presentación (Frontend):** componentes React, enrutado, estilos con TailwindCSS y gestión de estado de autenticación mediante contextos.

- **Capa de servicios (BaaS):** Supabase proporciona base de datos PostgreSQL, autenticación de usuarios, almacenamiento de ficheros (portadas y avatares) y políticas de seguridad a nivel de fila (Row Level Security).

La comunicación entre la aplicación y Supabase se realiza mediante la librería oficial @supabase/supabase-js, utilizando peticiones HTTP sobre HTTPS y respuestas en formato JSON.

En el siguiente esquema encontramos la estructura principal del proyecto:

```
bookflow/
├── public/              # Recursos estáticos (logos, imágenes por defecto)
├── docs/                # Documentación del proyecto
├── src/
│   ├── components/      # Layout.jsx, Navbar.jsx, BookCard.jsx, RequireAuth.js
│   ├── context/         # AuthContext.jsx
│   ├── lib/             # supabaseClient.js, googleBooksAPI.js
│   ├── pages/
│   │   ├── AdminCatálogo.jsx
│   │   ├── Explorar.jsx
│   │   ├── Home.jsx
│   │   ├── Login.jsx
│   │   ├── MiBiblioteca.jsx
│   │   ├── NotFound.jsx
│   │   └── Perfil.jsx
│   ├── App.jsx          # Enrutamiento principal
│   ├── index.css        # Estilos globales Tailwind
│   └── main.jsx         # Entrada de la aplicación React
├── .env.local           # Variables de entorno
├── .gitignore
├── package.json
└── tailwind.config.js
```

A continuación, se detalla la arquitectura diferenciando la parte de frontend y el backend (Supabase).

2.1 Frontend

2.1.1 Tecnologías usadas

- **Lenguaje principal:** JavaScript con JSX para la definición de componentes.

- **Framework / Librerías:**
  - **React:** construcción de la SPA mediante componentes funcionales y hooks.

  - **Vite:** herramienta de bundling y desarrollo rápido, que proporciona recarga en caliente y una configuración mínima.
  - **React Router:** gestión de las rutas públicas y protegidas (por ejemplo, /, /login, /explorar, /biblioteca, /admin).

  - **TailwindCSS:** sistema de utilidades CSS para maquetar la interfaz, dar soporte a modo oscuro y garantizar el diseño responsive.

  - **React Icons:** librería de iconos para mejorar la usabilidad y la comunicación visual en botones y menús.

2.1.2 Justificación de la elección

- React y Vite permiten un desarrollo muy ágil, con un ecosistema maduro y abundante documentación.

- TailwindCSS simplifica la gestión de estilos al trabajar con clases utilitarias, evitando archivos CSS muy grandes y facilitando la coherencia visual. Esto es vital para asegurar el correcto comportamiento multiplataforma (DAM).

- React Router se integra de forma natural con React y permite proteger rutas en función del estado de autenticación (evitando que usuarios no registrados accedan a la biblioteca).

2.1.3 Entorno de desarrollo

- **Sistema operativo:** Windows.

- **IDE:** Visual Studio Code, con extensiones para JavaScript, React, TailwindCSS e integración Git.

- **Gestor de dependencias:** npm (incluido con Node.js).

- **Navegador para pruebas:** Google Chrome, usando sus herramientas de desarrollo para depuración y análisis de rendimiento (modo dispositivo móvil activado).

- **Control de versiones:** Git + GitHub, donde se almacena el repositorio con el código fuente del proyecto. Este entorno se ha elegido por ser estándar en el desarrollo web moderno.

2.2 Backend

En este proyecto no se ha desarrollado un backend propio con un framework tradicional (tipo Spring o Laravel). En su lugar, se ha optado por un enfoque *Backend as a Service* utilizando Supabase.

2.2.1 Tecnologías usadas

- **Supabase:**
    - **PostgreSQL**, para almacenar datos de perfiles de usuario, catálogo de libros, reseñas y listas de biblioteca.
    - **Supabase Auth** para la autenticación (registro, login, logout) y gestión de sesiones seguras en el cliente.
    - **Supabase Storage** para el almacenamiento de avatares de usuario y carátulas personalizadas de libros.
    - **Row Level Security (RLS)** para asegurar que cada usuario solo pueda leer y modificar sus propias reseñas y listas de lectura.
- **Librería de acceso:**
    - @supabase/supabase-js para conectarse desde React al proyecto de Supabase utilizando la URL del servicio y la clave pública anónima.

2.2.2 Justificación

- Supabase proporciona en un único servicio todo lo necesario (BD, Auth, Storage), lo que reduce drásticamente el tiempo de desarrollo y la complejidad de despliegue.

- Al utilizar PostgreSQL se dispone de un sistema potente y estándar, ideal para gestionar las relaciones lógicas entre usuarios, libros y opiniones de forma robusta.

2.2.3 Entorno de desarrollo

- **Dashboard web de Supabase** para la creación de tablas, definición de políticas RLS y ejecución de scripts SQL.

- **Vercel** como plataforma de despliegue del frontend, configurada con las variables de entorno necesarias para conectar con Supabase en producción.

3. Documentación Técnica

3.1 Análisis

En esta sección se resumen los principales requisitos funcionales y no funcionales que han guiado el desarrollo de BookFlow.

3.1.1 Roles principales

- **Invitado:** puede acceder a la página de inicio, explorar el catálogo público de libros, leer reseñas de otros usuarios y acceder al formulario de registro/login.

- **Lector Registrado:** dispone de perfil, panel de control de su biblioteca personal (con estados de lectura) y capacidad para redactar, editar y eliminar sus propias reseñas.

- **Administrador:** puede acceder a un panel privado para gestionar el catálogo global (CRUD de libros) y administrar las cuentas de los usuarios (listar, filtrar y activar/desactivar).

3.1.2 Requisitos funcionales

- **RF1:** Un invitado puede registrarse mediante email y contraseña, confirmar su cuenta y convertirse en usuario registrado mediante Supabase Auth.

- **RF2:** Un usuario registrado puede iniciar y cerrar sesión de forma segura y recuperar la contraseña mediante correo electrónico.

- **RF3:** El usuario puede gestionar su perfil, incluyendo datos personales básicos, biografía y actualizar su avatar almacenado en Supabase Storage.

- **RF4:** El usuario dispone de un catálogo interactivo con buscador y filtros (por género o autor) para explorar obras literarias.

- **RF5:** Desde la sección "Mi Biblioteca", el usuario puede añadir, consultar, actualizar el estado (Pendiente, Leyendo, Terminado) y eliminar libros de sus listas personales.

- **RF6:** El usuario puede crear, editar y borrar reseñas textuales y puntuaciones (1 a 5 estrellas) de los libros que ha leído.

- **RF7:** El usuario puede generar y descargar un informe en PDF de su lista de libros leídos.

- **RF8:** El administrador dispone de un panel exclusivo para dar de alta nuevos libros, modificarlos o eliminarlos del sistema.

- **RF9:** El administrador puede consultar la lista de usuarios, filtrarla, y activar/desactivar cuentas sin eliminarlas físicamente.

3.1.3 Requisitos no funcionales

- **RNF1. Arquitectura:** aplicación *front-only*. La aplicación puede desplegarse fácilmente en servicios de hosting estáticos y no requiere mantenimiento de servidores propios.

- **RNF2. Seguridad:** autenticación y control de acceso con Supabase Auth. Solo los usuarios autenticados acceden a sus datos personales (listas y perfil) y ningún usuario puede modificar reseñas o bibliotecas de otros mediante el uso de Row Level Security (RLS).

- **RNF3. Privacidad:** Los datos del usuario se utilizan exclusivamente para la gestión de su perfil literario. El usuario puede eliminar su cuenta, lo cual implica la desaparición del perfil y restricción de acceso a sus registros asociados.

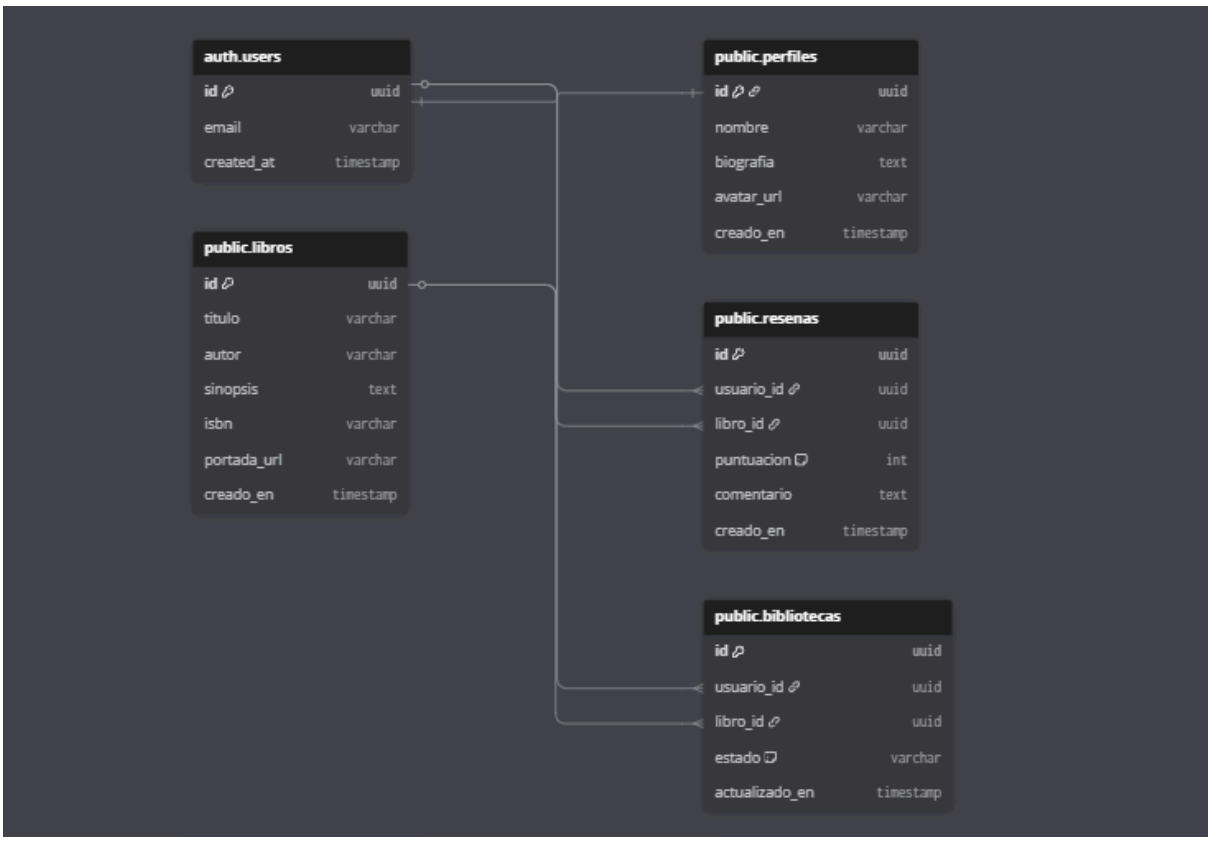
- **RNF4. Rendimiento:** fluidez en la experiencia de usuario. Al tratarse de una SPA, la navegación entre el catálogo y la biblioteca es inmediata.
- RNF5. Interfaz Multiplataforma (DAW/DAM): La aplicación utiliza TailwindCSS para ofrecer un diseño limpio, moderno y totalmente *responsive*, garantizando una experiencia de uso similar a una app nativa en dispositivos móviles.

- RNF6. Generación de PDF: Los usuarios pueden descargar sus reportes en PDF directamente desde el cliente web, sin necesidad de procesamiento en el backend.

3.2 Desarrollo

3.2.1 Base de Datos

La aplicación BookFlow utiliza una base de datos PostgreSQL gestionada por Supabase. La estructura de tablas se ha diseñado teniendo en cuenta la integración directa con el sistema de autenticación Supabase Auth, lo cual influye en la forma en que se representan las relaciones dentro del modelo.



La base de datos se compone de cuatro entidades principales en el esquema público:

- **perfiles:** Tiene una relación 1:1 con auth.users (comparten el mismo UUID como clave primaria y foránea) y guarda los datos públicos del lector.

- **libros:** Almacena el catálogo de obras con sus metadatos (título, autor, sinopsis, ISBN, URL de la portada).

- **resenas:** Mantiene una relación N:1 con auth.users (el autor de la reseña) y N:1 con la tabla libros. Guarda el comentario y la puntuación numérica.
- **bibliotecas:** Mantiene una relación N:1 con auth.users y N:1 con la tabla libros. Funciona como tabla puente para gestionar el estado de lectura ("Pendiente", "Leyendo", "Terminado").

Aunque en las herramientas visuales de diseño a veces las tablas del esquema público parezcan desconectadas de la autenticación, el vínculo lógico existe: las tablas de usuario referencian directamente a auth.users.id.

3.2.2 Justificación del diseño

El uso de auth.users como tabla raíz para la identidad ofrece ventajas importantes:

- Integración natural con Supabase Auth: un único origen de verdad para la identidad del usuario.

- Mayor seguridad: las políticas RLS pueden comparar fácilmente auth.uid() con los campos usuario_id de las reseñas y bibliotecas.

- Menor redundancia y evolución más sencilla del perfil sin afectar a otras tablas. Este enfoque es el recomendado por Supabase para aplicaciones *front-only*.

3.2.3 Diagrama de casos de uso

Los casos de uso representan las funcionalidades que la aplicación implementa realmente:

- **Registro e inicio de sesión:** actores (usuario y admin) pueden autenticarse.

- **Gestión de perfil:** el usuario puede editar sus datos y avatar.

- **Exploración de catálogo:** cualquier actor puede buscar y visualizar detalles de los libros.

- **Gestión de Biblioteca:** el lector añade libros a sus listas y actualiza su estado.

- **Sistema de Reseñas:** el lector valora y comenta los libros que ha leído.

- **Descarga en PDF:** exportación de la lista de lectura utilizando jsPDF + html2canvas.

- **Administración del sistema:** el administrador gestiona el CRUD del catálogo de libros y el estado de las cuentas de usuario.

3.3 Pruebas realizadas

Se han llevado a cabo pruebas manuales sistemáticas sobre las funcionalidades principales:

- **Pruebas de registro y login:** Comprobación de casos válidos (email/contraseña correctos) e inválidos (formatos incorrectos), asegurando la correcta redirección y manejo de errores.

- **Pruebas de navegación y adaptabilidad:** Verificación del diseño *responsive* en distintas resoluciones (escritorio, tablet, móvil) para asegurar el cumplimiento multiplataforma.

- **Pruebas del CRUD de libros (Admin):** Creación de libros con distintos datos, edición de portadas y eliminación, comprobando el reflejo inmediato en la base de datos y el catálogo.

- **Pruebas de Biblioteca y Reseñas:** Cambio de estados en los libros de un usuario y publicación de reseñas, verificando que las políticas RLS bloquean la modificación de datos por parte de otros usuarios no autorizados.

- **Pruebas del panel de administración de usuarios:** Verificación de filtros por estado y activación/desactivación de cuentas.

4. Proceso de despliegue

El despliegue de **BookFlow** se realiza sobre dos servicios principales:

- **Supabase:** para la base de datos PostgreSQL, autenticación de usuarios y almacenamiento de imágenes.

- **Vercel:** para alojar el frontend construido con React + Vite.

4.1 Software necesario

- Navegador web moderno.

- Cuenta de desarrollador en GitHub.

- Cuenta en Supabase.

- Cuenta en Vercel.

- Node.js y npm instalados localmente para la ejecución y pruebas en el entorno de desarrollo.

4.2 Pasos para desplegar la aplicación

4.2.1 Configurar Supabase

1. Crear un nuevo proyecto en el panel de control de Supabase.

2. Definir las tablas perfiles, libros, resenas y bibliotecas mediante el editor SQL o el diseñador visual de tablas.

3. Establecer las políticas de seguridad (RLS) necesarias para garantizar que cada usuario solo pueda acceder y modificar sus listas y reseñas privadas.

4. Habilitar Supabase Auth con el proveedor de correo electrónico para el registro de usuarios.
5. Crear los *buckets* en Supabase Storage para alojar los avatares y las portadas de los libros.

6. Obtener la URL del proyecto y la clave pública (anon key), que se utilizarán para conectar el frontend.

4.2.2 Configurar el proyecto local

1. Clonar el repositorio desde GitHub:
   Bash
   ```
   git clone https://github.com/tu-usuario/bookflow.git

   cd bookflow
   ```
2. Crear un archivo .env.local en la raíz del proyecto con las variables de entorno de conexión:
   Fragmento de código
   ```
   VITE_SUPABASE_URL=tu_url_de_supabase_aqui

   VITE_SUPABASE_ANON_KEY=tu_anon_key_aqui
   ```
3. Instalar las dependencias de Node.js y arrancar el servidor local:
   Bash
   ```
   npm install

   npm run dev
   ```

4.2.3 Despliegue en Vercel

1. Iniciar sesión en Vercel y elegir la opción de "Importar proyecto desde GitHub".
2. Seleccionar el repositorio de BookFlow.
3. En el apartado de configuración, añadir las variables de entorno (VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY) con los mismos valores utilizados en el entorno local.

4. Lanzar el despliegue.

5. Una vez finalizado el proceso de *build*, Vercel proporcionará una URL pública (con certificado HTTPS automático) desde la que se puede acceder a la aplicación web. Cualquier nuevo *commit* que se suba a la rama principal (main) del repositorio volverá a desplegarse de forma automática gracias a la integración continua con Vercel.

5. Propuestas de mejoras

Durante la fase de diseño y desarrollo del proyecto se han identificado distintas posibilidades de mejora para futuras iteraciones, tanto a nivel funcional como técnico:

- **Integración de APIs externas:** Conectar la aplicación con la API de *Google Books* o *Open Library* para autocompletar la información del libro (sinopsis, autor, número de páginas y portada) simplemente introduciendo el ISBN.

- **Gamificación y Retos de lectura:** Implementar un sistema de "Reto Anual", donde el usuario establezca una meta de libros a leer durante el año, mostrando una barra de progreso visual en su panel de control.

- **Componente social avanzado:** Añadir la funcionalidad de "Seguir" a otros usuarios para poder ver un muro de actividad (*feed*) con las últimas lecturas y reseñas de la comunidad, fomentando la interacción.

- **Clubes de lectura virtuales:** Crear espacios (foros o chats) dentro de la aplicación asociados a un libro específico, para que los lectores puedan debatir en tiempo real.

- **Notificaciones push o por email:** Avisar a los usuarios registrados mediante notificaciones automáticas cuando un autor de su biblioteca publique una nueva obra, o cuando sus reseñas reciban una interacción ("Me gusta").